

Lab 2

Sam Kutsyn, 2581500

EE 446

July 8, 2026

Task 3

Confirmed blink is working.

Task 4

4.a Serial Output

```
void setup() {  
    Serial.begin(115200);  
    delay(1500);  
    Serial.println("Serial test started");  
}  
  
void loop() {  
    static int count = 0;  
    Serial.print("Count: ");  
    Serial.println(count);  
    count++;  
    delay(1000);  
}
```

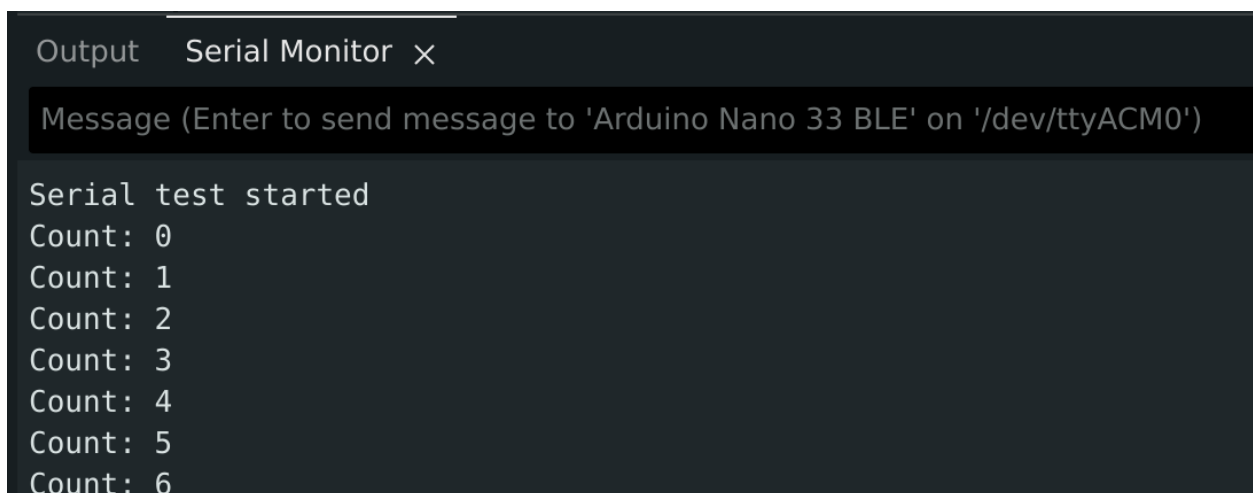


Figure 1: Serial Monitor for Task 4.

4.a Plotter Output

```
void setup() {  
    Serial.begin(115200);  
    delay(1500);  
}  
  
void loop() {  
    static int x = 0;  
    int y = (x % 20);  
    Serial.println(y);  
    x++;  
    delay(100);  
}
```

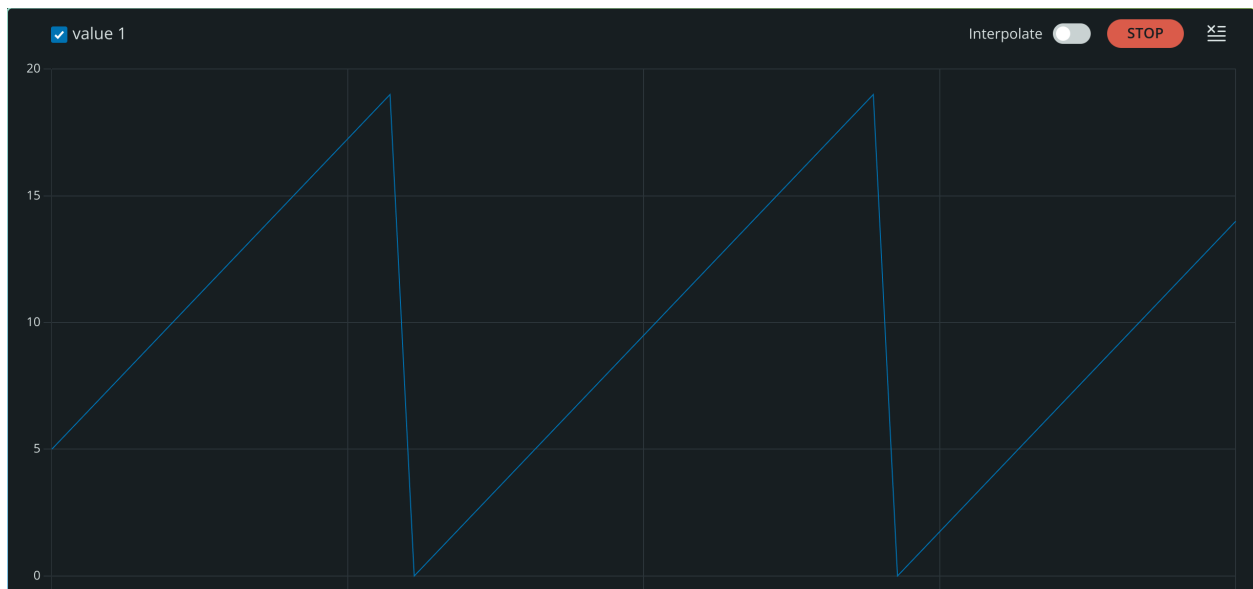


Figure 2: Serial Plotter for Task 4.

Task 5

```
#include <PDM.h>  
  
short sampleBuffer[256];  
volatile int samplesRead = 0;  
  
void onPDMdata() {  
    PDM.available();  
    int bytesAvailable;  
    PDM.read(sampleBuffer, bytesAvailable);  
    samplesRead = bytesAvailable / 2;  
}
```

```

void setup() {
  Serial.begin(115200);
  delay(1500);
  PDM.onReceive(onPDMdata);
  if (!PDM.begin(1, 16000)) {
    Serial.println("Failed to start PDM microphone.");
    while (1)
      ;
  }
}

```

```

void loop() {
  if (samplesRead) {
    long sum = 0;
    for (int i = 0; i < samplesRead; i++) {
      sum += abs(sampleBuffer[i]);
    }
    int level = sum / samplesRead;
    Serial.println(level);
    samplesRead = 0;
  }
}

```

![[Insert screenshot of Serial Plotter (quiet)]]

![[Insert screenshot of Serial Plotter (active)]]

Singing caused the most prominent change.

Task 6

6.a IMU Verification

```
#include <Arduino_BMI270_BMM150.h>
```

```

void setup() {
  Serial.begin(115200);
  delay(1500);
  if (!IMU.begin()) {
    Serial.println("Failed to initialize IMU.");
    while (1)
      ;
  }
  Serial.println("Accelerometer test started");
  Serial.println("ax,ay,az");
}

```

```

void loop() {
  float x, y, z;
  if (IMU.accelerationAvailable()) {

```

```

    IMU.readAcceleration(x, vy, z);
    Serial.print(x, 3);
    Serial.print(",");
    Serial.print(y, 3);
    Serial.print(",");
    Serial.println(z, 3);
}
delay(100);
}

```

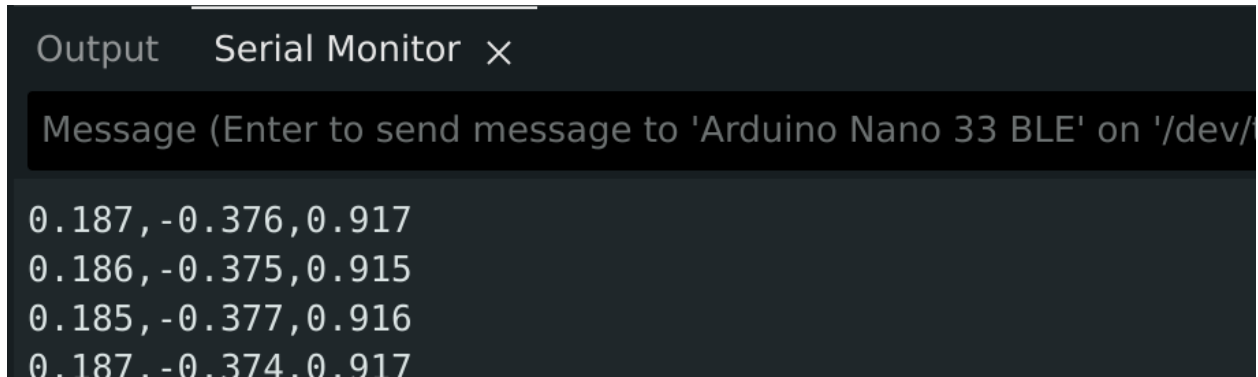


Figure 3: Serial Monitor for Task 6a.

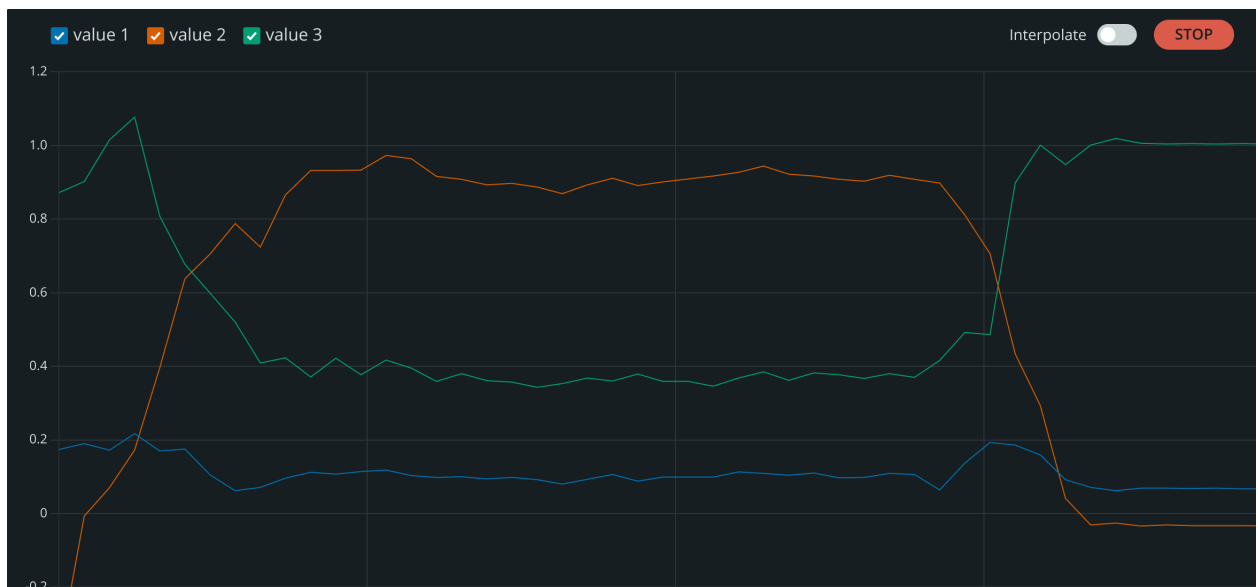


Figure 4: Serial Plotter for Task 6a.

Rotating the board produced the clearest change.

6.b Gyroscope Verification

```
#include <Arduino_BMI270_BMM150.h>

void setup() {
  Serial.begin(115200);
  delay(1500);
  if (!IMU.begin()) {
    Serial.println("Failed to initialize IMU.");
    while (1)
      ;
  }
  Serial.println("Gyroscope test started");
  Serial.println("gx,gy,gz");
}

void loop() {
  float x, y, z;
  if (IMU.gyroscopeAvailable()) {
    IMU.readGyroscope(x, y, z);
    Serial.print(x, 3);
    Serial.print(",");
    Serial.print(y, 3);
    Serial.print(",");
    Serial.println(z, 3);
  }
  delay(100);
}
```

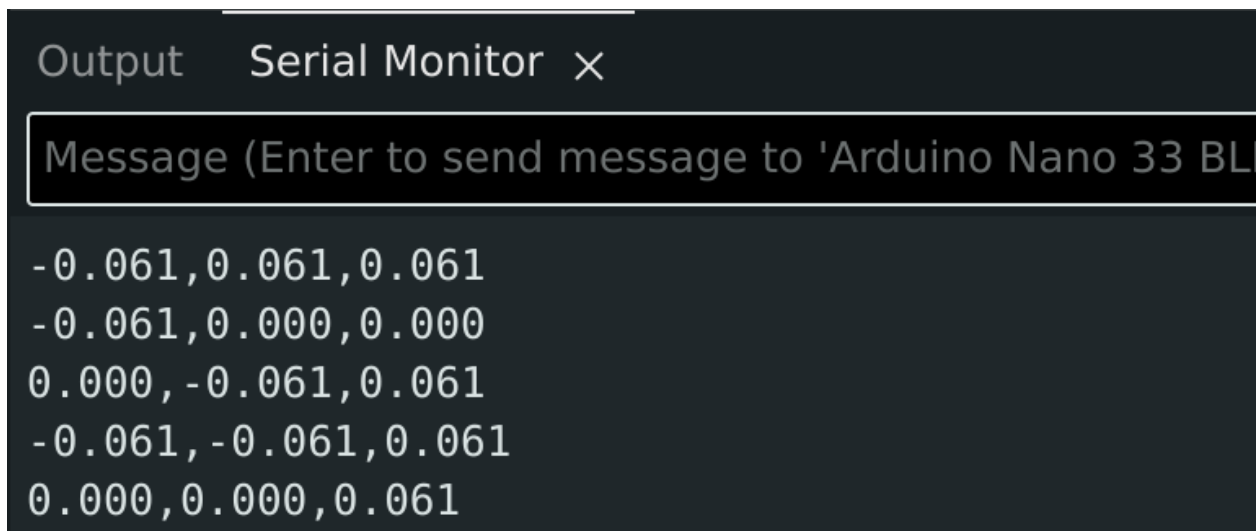


Figure 5: Serial Monitor for Task 6b.



Figure 6: Serial Plotter for Task 6b.

Continuously rotating the board produced the clearest change.

6.c Magnetometer Verification

```
#include <Arduino_BMI270_BMM150.h>

void setup() {
  Serial.begin(115200);
  delay(1500);
  if (!IMU.begin()) {
    Serial.println("Failed to initialize IMU.");
    while (1)
      ;
  }
  Serial.println("Magnetometer test started");
  Serial.println("mx,my,mz");
}

void loop() {
  float x, y, z;
  if (IMU.magneticFieldAvailable()) {
    IMU.readMagneticField(x, y, z);
    Serial.print(x, 3);
  }
}
```

```

    Serial.print(",");
    Serial.print(y, 3);
    Serial.print(",");
    Serial.println(z, 3);
}
delay(100);
}

```

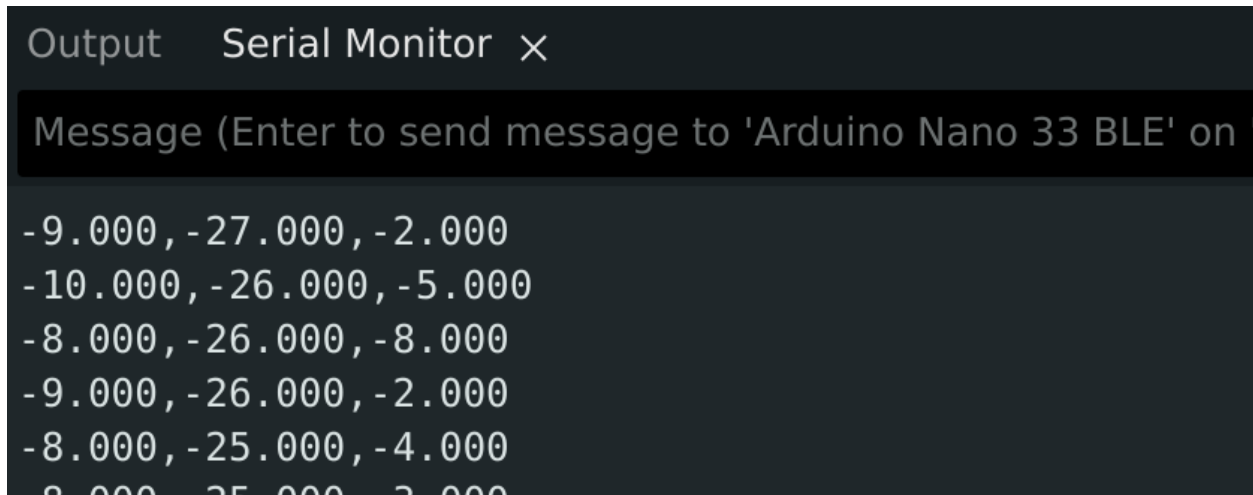


Figure 7: Serial Monitor for Task 6b.

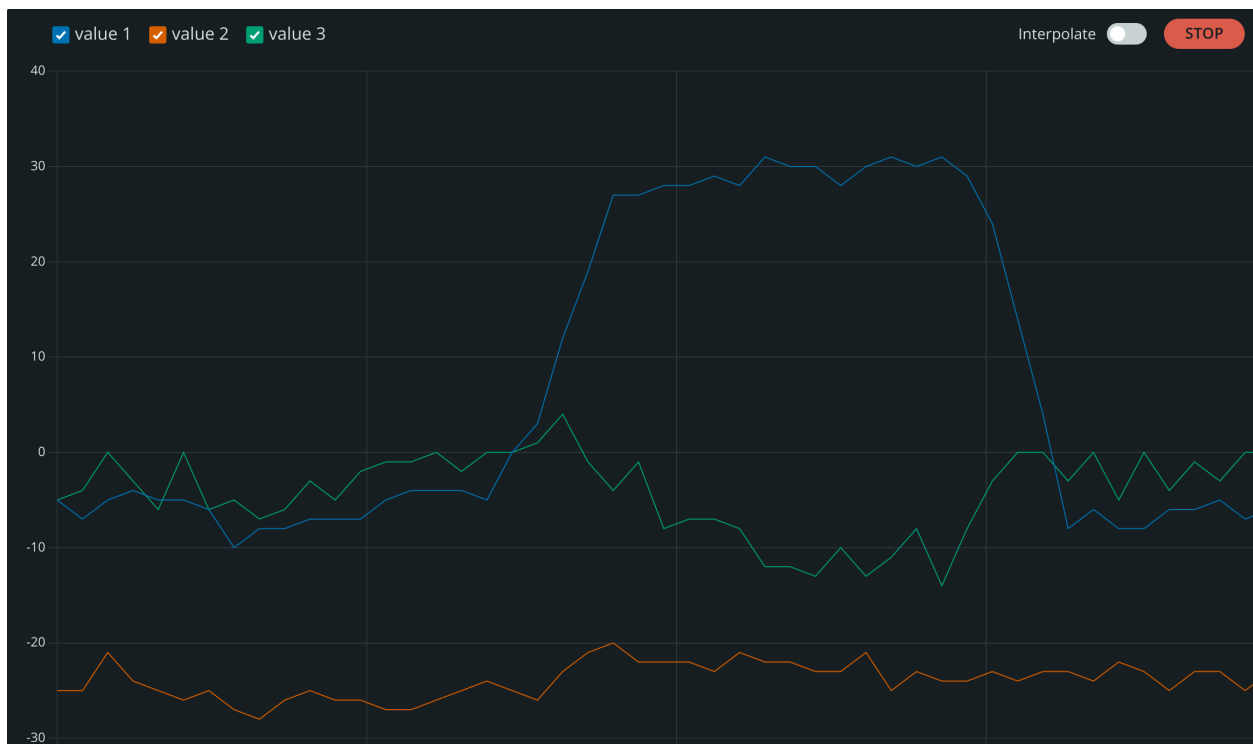


Figure 8: Serial Plotter for Task 6b.

Continuously rotating the board produced the clearest change.

Task 7 Humidity and Temperature Verification

```
#include <Arduino_HS300x.h>

void setup() {
  Serial.begin(115200);
  delay(1500);
  if (!HS300x.begin()) {
    Serial.println("Failed to initialize humidity/temperature sensor.");
    while (1)
      ;
  }
  Serial.println("Humidity and temperature test started");
}

void loop() {
  float temperature = HS300x.readTemperature();
  float humidity = HS300x.readHumidity();
  Serial.print("Temperature (C): ");
  Serial.print(temperature, 2);
  Serial.print(" | Humidity (%): ");
  Serial.println(humidity, 2);
  delay(1000);
}
```

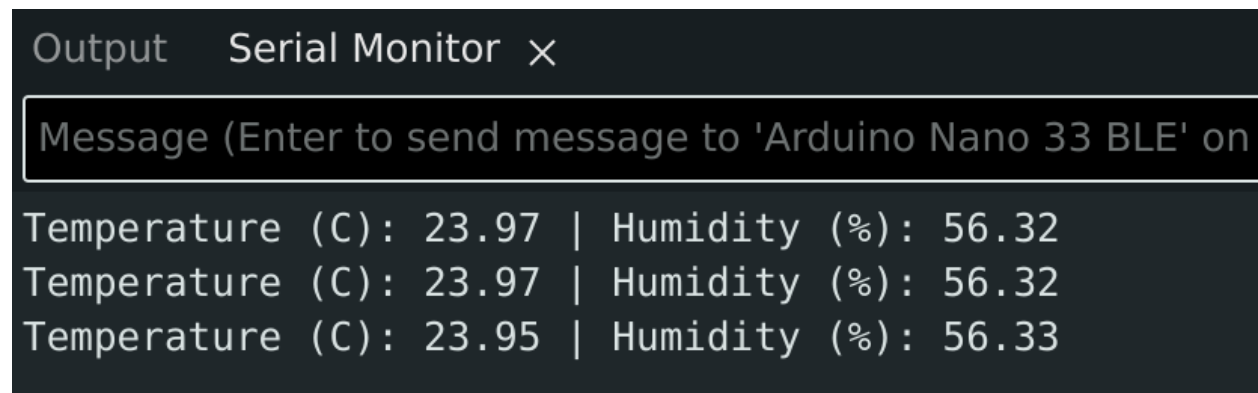


Figure 9: Serial Monitor for Task 7.

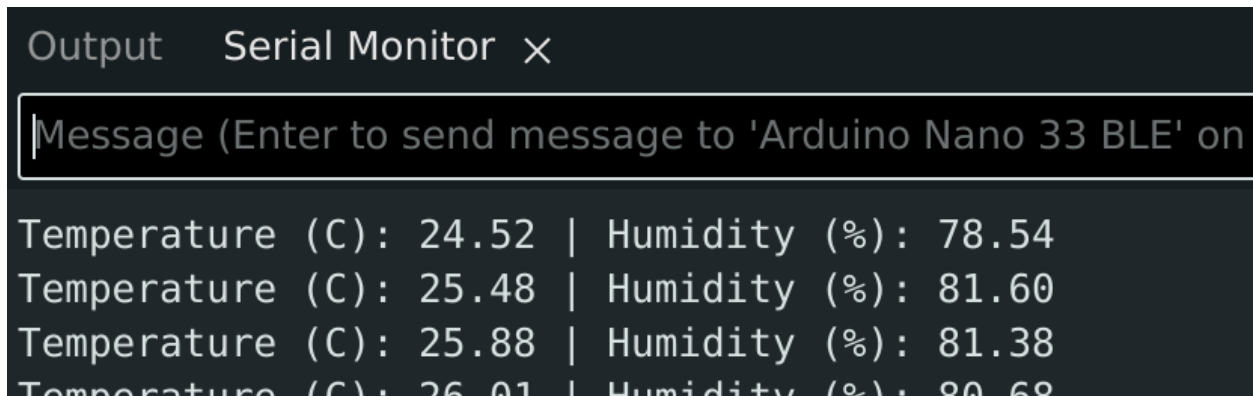


Figure 10: Serial Plotter for Task 7, changed.

Putting my hand over the board produced the clearest change.

Task 8 Barometric Pressure Verification

```
#include <Arduino_LPS22HB.h>

void setup() {
  Serial.begin(115200);
  delay(1500);
  if (!BARO.begin()) {
    Serial.println("Failed to initialize barometric pressure sensor.");
    while (1)
      ;
  }
  Serial.println("Barometric pressure test started");
}

void loop() {
  float pressure = BARO.readPressure();
  float temperature = BARO.readTemperature();
  Serial.print("Pressure (kPa): ");
  Serial.print(pressure, 3);
  Serial.print(" | Temperature (C): ");
  Serial.println(temperature, 2);
  delay(1000);
}
```

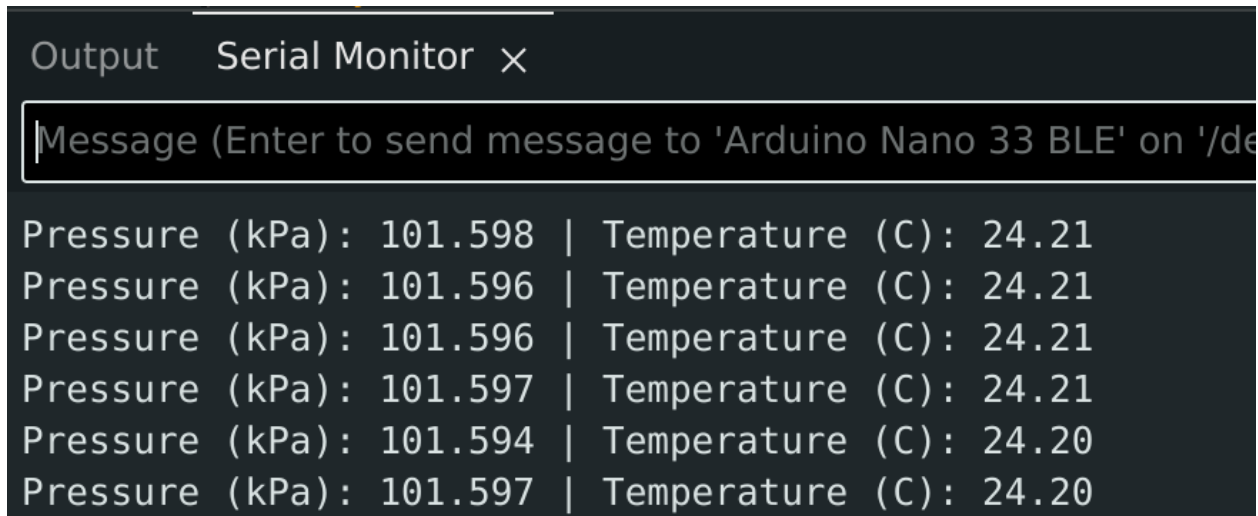


Figure 11: Serial Monitor for Task 8.

Pressure readings are stable at ≈ 101.597 kPa

Task 9

9.a Proximity Verification

```
#include <Arduino_APDS9960.h>

void setup() {
  Serial.begin(115200);
  delay(1500);
  if (!APDS.begin()) {
    Serial.println("Failed to initialize APDS9960 sensor.");
    while (1)
      ;
  }
  Serial.println("Proximity test started");
}

void loop() {
  if (APDS.proximityAvailable()) {
    int proximity = APDS.readProximity();
    Serial.print("Proximity: ");
    Serial.println(proximity);
  }
  delay(100);
}
```

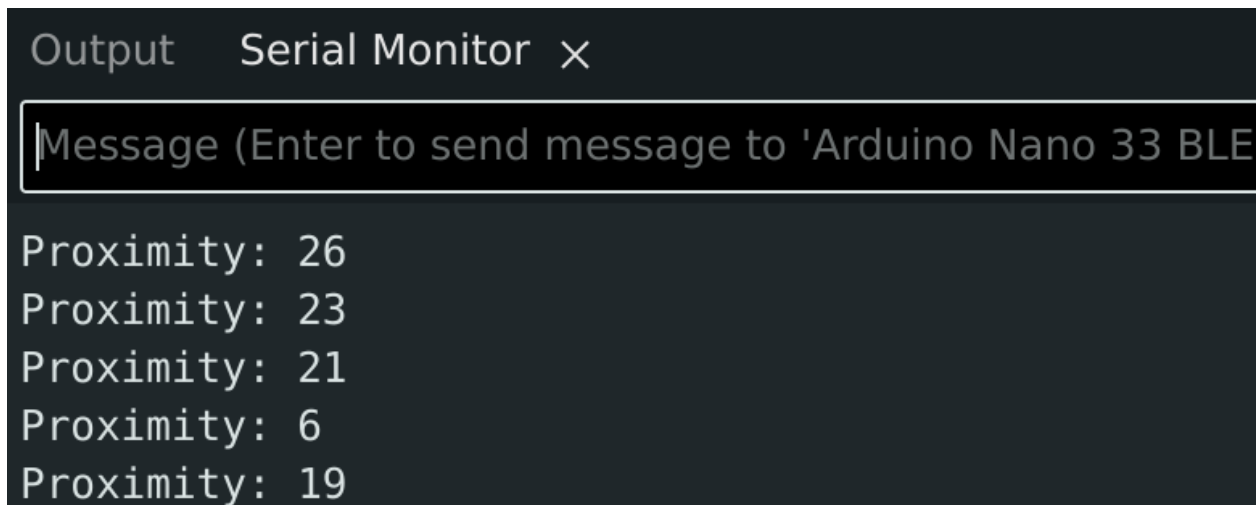


Figure 12: Serial Monitor for Task 9a, close.

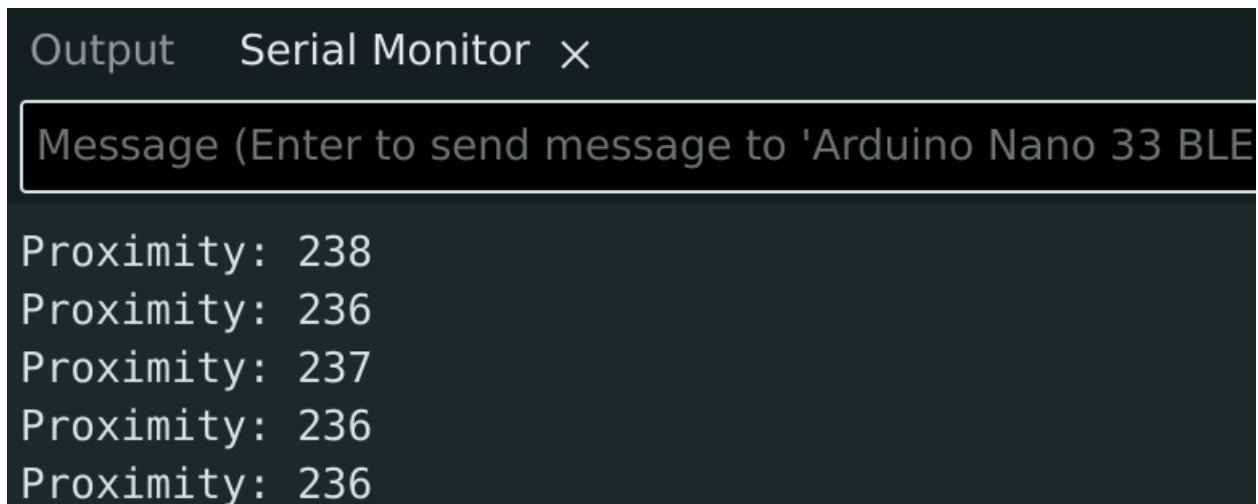


Figure 13: Serial Monitor for Task 9a, far.

Values changed from 0 to ≈ 239 depending on the distance of the hand from the board (ranging from 1cm to completely removed).

9.b Gesture Verification

```
#include <Arduino_APDS9960.h>

void setup() {
  Serial.begin(115200);
  delay(1500);
  if (!APDS.begin()) {
    Serial.println("Failed to initialize APDS9960 sensor.");
    while (1)
```

```

    ;
}
Serial.println("Gesture test started");
}

void loop() {
    if (APDS.gestureAvailable()) {
        int gesture = APDS.readGesture();
        if (gesture == GESTURE_UP) {
            Serial.println("UP");
        } else if (gesture == GESTURE_DOWN) {
            Serial.println("DOWN");
        } else if (gesture == GESTURE_LEFT) {
            Serial.println("LEFT");
        } else if (gesture == GESTURE_RIGHT) {
            Serial.println("RIGHT");
        }
    }
}
}

```

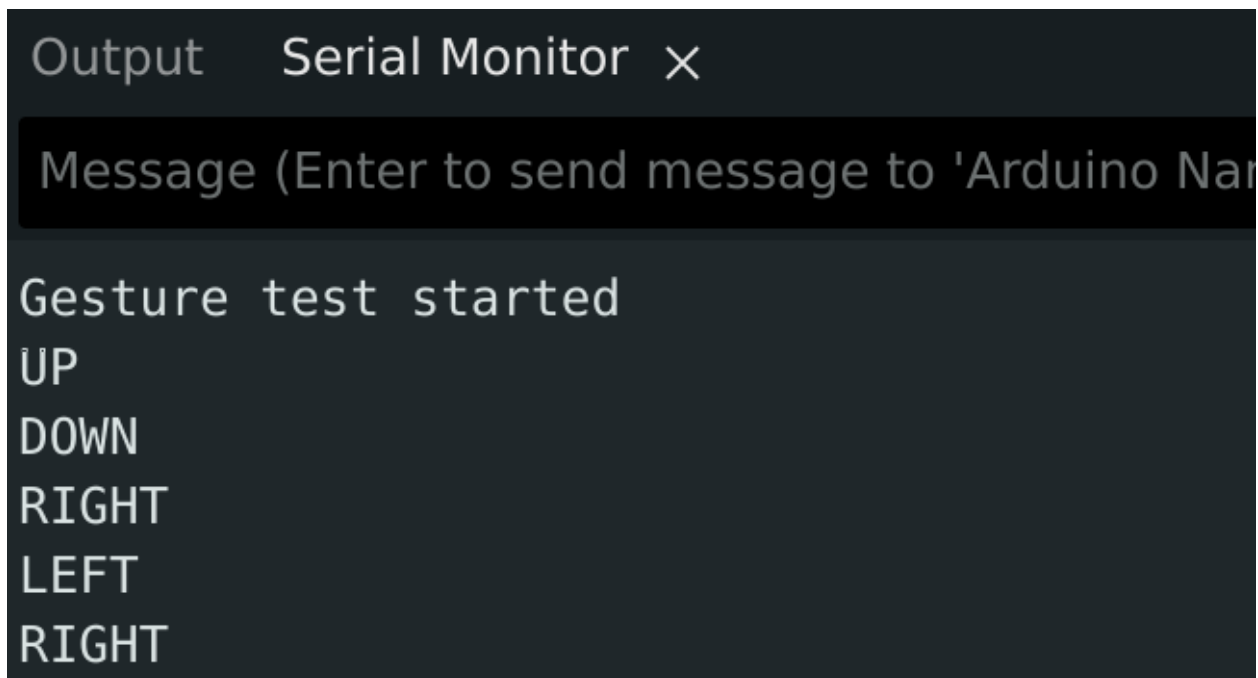


Figure 14: Serial Monitor for Task 9b.

Right/left gesture recognition was the most reliable.

9.c Ambient Light and RGB Color Verification

```
#include <Arduino_APDS9960.h>
```

```

void setup() {
  Serial.begin(115200);
  delay(1500);
  if (!APDS.begin()) {
    Serial.println("Failed to initialize APDS9960 sensor.");
    while (1)
      ;
  }
  Serial.println("Ambient light and color test started");
  Serial.println("r,g,b,clear");
}

void loop() {
  int r, g, b, c;
  if (APDS.colorAvailable()) {
    APDS.readColor(r, g, b, c);
    Serial.print(r);
    Serial.print(", ");
    Serial.print(g);
    Serial.print(", ");
    Serial.print(b);
    Serial.print(", ");
    Serial.print(c);
    Serial.println();
  }
  delay(200);
}

```

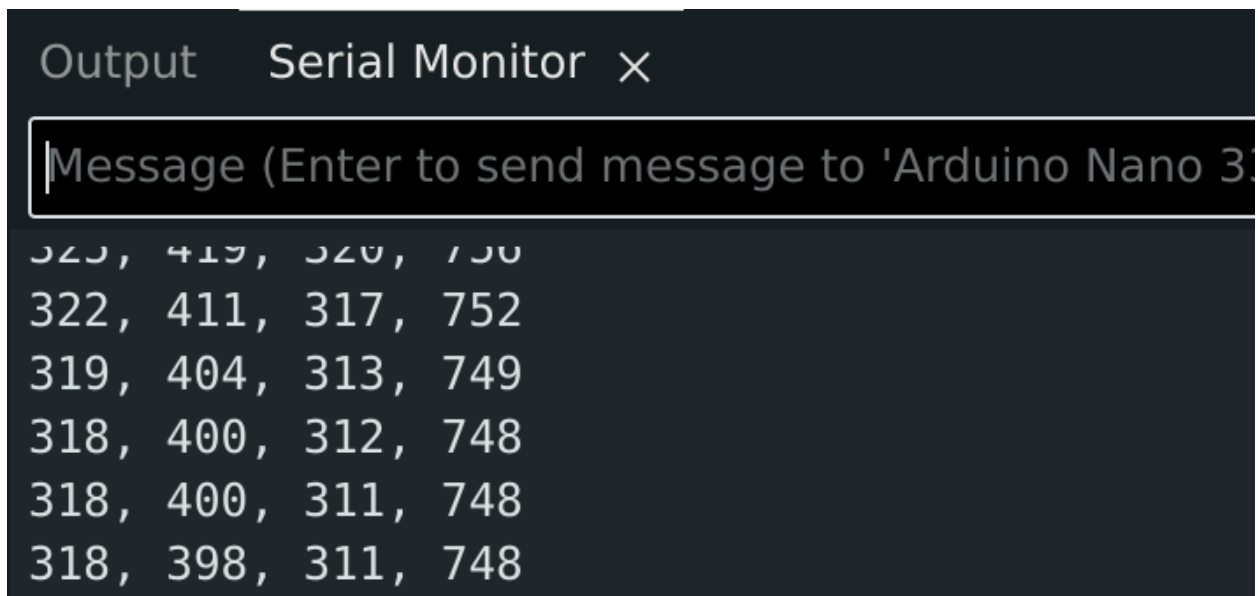


Figure 15: Serial Monitor for Task 9c, room light.

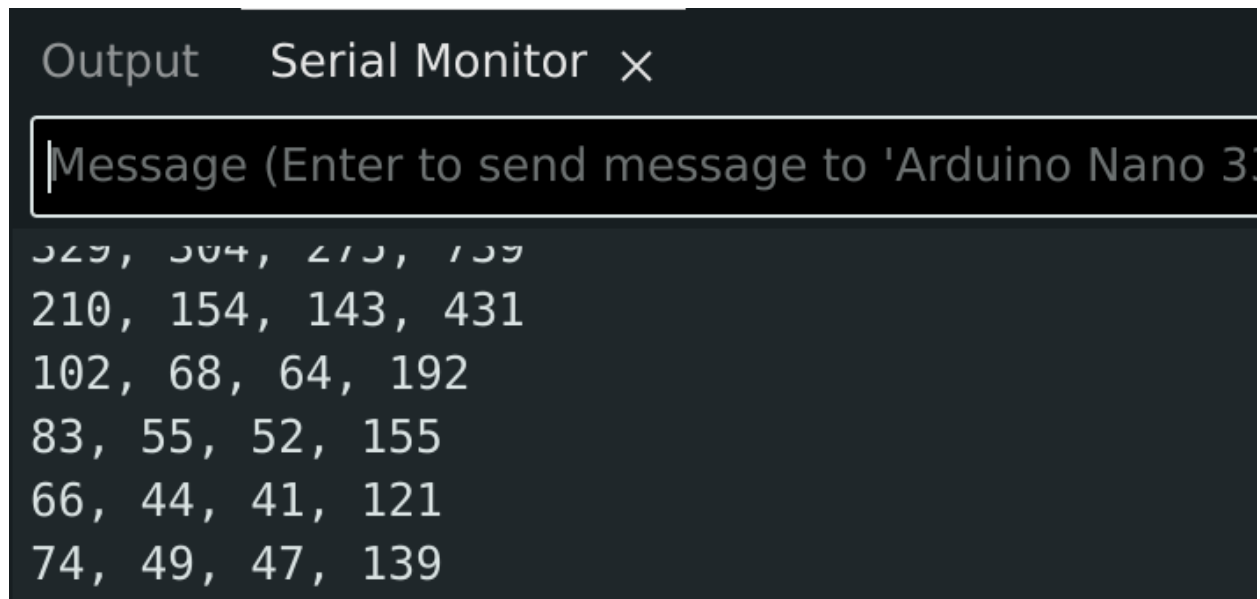


Figure 16: Serial Monitor for Task 9c, hand covered.

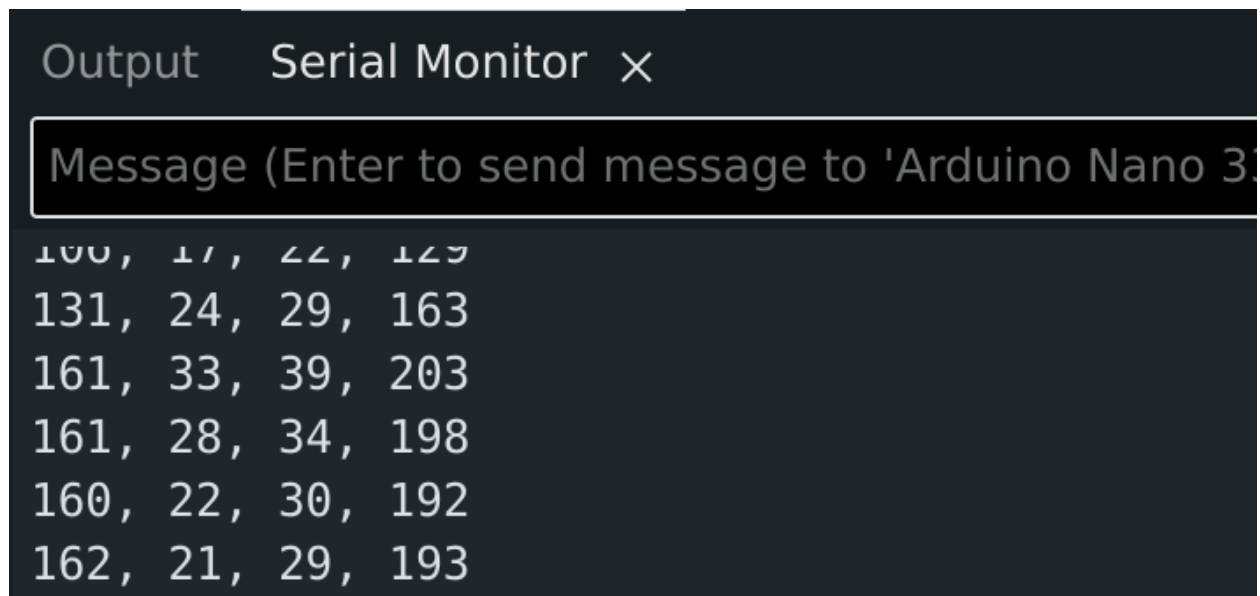


Figure 17: Serial Monitor for Task 9c, red light.

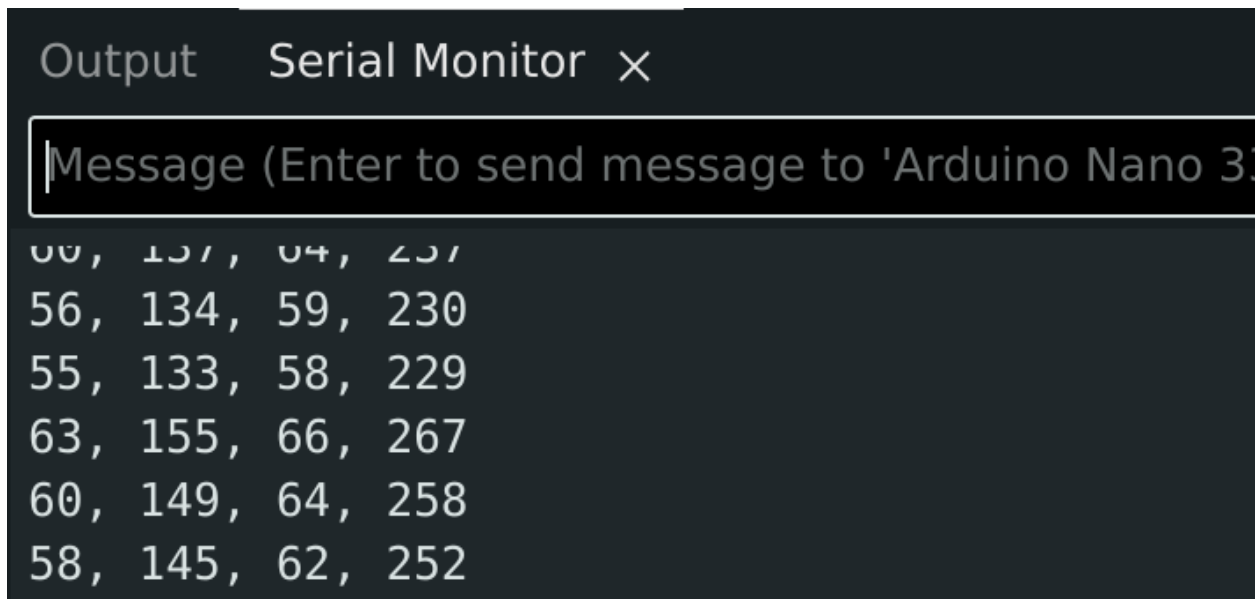


Figure 18: Serial Monitor for Task 9c, green light.

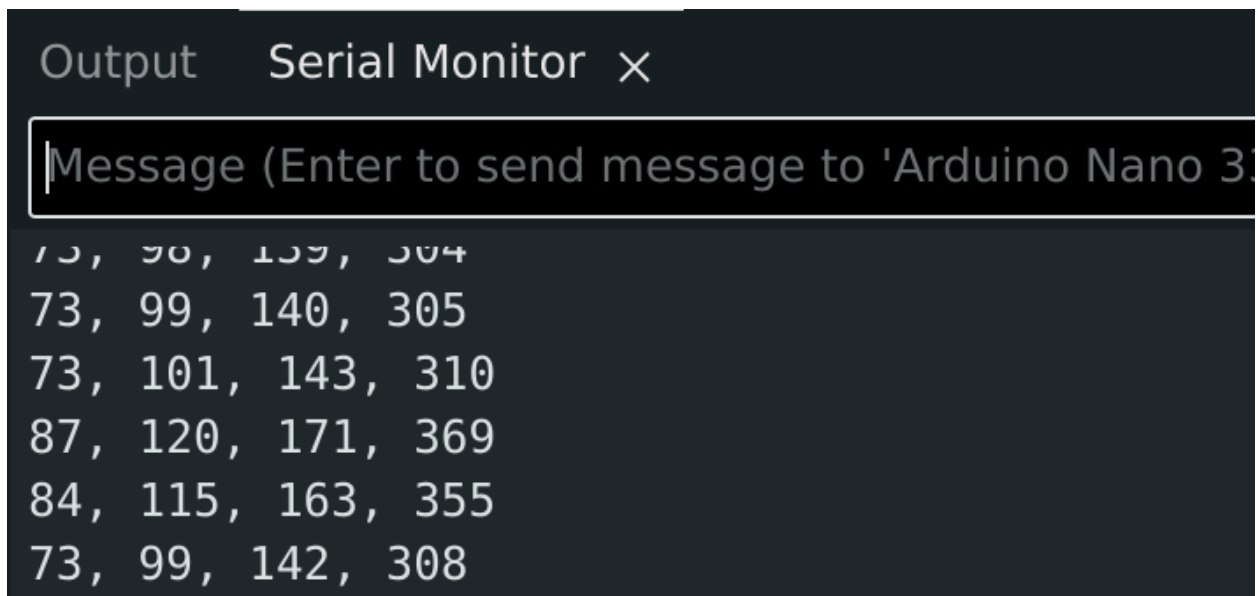


Figure 19: Serial Monitor for Task 9c, blue light.

Covering the sensor produced the clearest change.

Task 10: Smart Workspace Situation Classifier

See code at <https://gist.github.com/v-kut/99438100cac19f833893cd5db045b7a5>.

I decided to use two helper structs to make the code a little cleaner:

```
struct SensorReadings {
```

```

    int mic;
    int light;
    float motion;
    int proximity;
};

struct SituationFlags {
    int sound;
    int dark;
    int moving;
    int near;
};

```

These structs are passed to the classifier function, which performs simple if checks to decide the state:

```

String classify(const SituationFlags &f) {
    if (f.moving && f.near && f.sound && !f.dark) {
        return "NOISY_BRIGHT_MOVING_NEAR";
    }
    if (!f.moving && f.near && !f.sound && f.dark) {
        return "QUIET_DARK_STEADY_NEAR";
    }
    if (!f.moving && !f.near && f.sound && !f.dark) {
        return "NOISY_BRIGHT_STEADY_FAR";
    }
    if (!f.moving && !f.near && !f.sound && !f.dark) {
        return "QUIET_BRIGHT_STEADY_FAR";
    }

    if (f.near && f.moving) return "NOISY_BRIGHT_MOVING_NEAR";
    if (f.near && f.dark) return "QUIET_DARK_STEADY_NEAR";
    if (f.sound) return "NOISY_BRIGHT_STEADY_FAR";
    return "QUIET_BRIGHT_STEADY_FAR";
}

```

The thresholds were selected by uhhhhhhh... experimenting. There is no calibration stage so they should be adjusted for every environment they are in.


```
Output  Serial Monitor X
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/ttyACM0')

raw,mic=5,clear=7,motion=0.0010,prox=0
flags,sound=0,dark=1,moving=0,near=1
state,QUIET_DARK_STEADY_NEAR

raw,mic=55,clear=7,motion=0.0000,prox=0
flags,sound=0,dark=1,moving=0,near=1
state,QUIET_DARK_STEADY_NEAR

raw,mic=13,clear=7,motion=0.0038,prox=0
flags,sound=0,dark=1,moving=0,near=1
state,QUIET_DARK_STEADY_NEAR
```

Figure 20: Serial Monitor for Task 10, *QUIET_BRIGTH_STEADY_NEAR*.

```
Output  Serial Monitor X
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/ttyACM0')

raw,mic=15,clear=108,motion=0.0038,prox=237
flags,sound=0,dark=1,moving=0,near=0
state,QUIET_BRIGTH_STEADY_FAR

raw,mic=4,clear=109,motion=0.0005,prox=241
flags,sound=0,dark=1,moving=0,near=0
state,QUIET_BRIGTH_STEADY_FAR

raw,mic=16,clear=109,motion=0.0008,prox=241
flags,sound=0,dark=1,moving=0,near=0
state,QUIET_BRIGTH_STEADY_FAR
```

Figure 21: Serial Monitor for Task 10, *QUIET_BRIGTH_STEADY_FAR*.

```
Output  Serial Monitor  X
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/ttyACM0')

raw,mic=4,clear=107,motion=0.0030,prox=235
flags,sound=0,dark=1,moving=0,near=0
state,QUIET_BRIGHT_STEADY_FAR

raw,mic=225,clear=107,motion=0.0018,prox=240
flags,sound=1,dark=1,moving=0,near=0
state,NOISY_BRIGHT_STEADY_FAR

raw,mic=139,clear=107,motion=0.0018,prox=240
flags,sound=1,dark=1,moving=0,near=0
state,NOISY_BRIGHT_STEADY_FAR
```

Figure 22: Serial Monitor for Task 10, *NOISY_BRIGHT_STEADY_FAR.png*.

```
Output  Serial Monitor  X
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/ttyACM0')

raw,mic=33,clear=3,motion=0.0744,prox=0
flags,sound=0,dark=1,moving=1,near=1
state,NOISY_BRIGHT_MOVING_NEAR

raw,mic=168,clear=3,motion=0.0765,prox=0
flags,sound=1,dark=1,moving=1,near=1
state,NOISY_BRIGHT_MOVING_NEAR

raw,mic=59,clear=2,motion=0.0528,prox=0
flags,sound=0,dark=1,moving=1,near=1
state,NOISY_BRIGHT_MOVING_NEAR
```

Figure 23: Serial Monitor for Task 10, *NOISY_BRIGHT_MOVING_NEAR*.

Task 11 Rule-Based Environmental Monitoring

See code at <https://gist.github.com/v-kut/99438100cac19f833893cd5db045b7a5>.

Same story as before - helper structs for cleaner code:

```
struct SensorReadings {
    float rh;
    float temp;
    float mag;
    int r;
    int g;
    int b;
    int c;
};
```

```
struct EventFlags {
    int humid;
    int temp;
    int mag;
    int light;
};
```

This time, I also keep track of the last time the event changed, which is then used for debouncing

```
unsigned long eventStartTimestamp = 0;
```

```
void loop() {
    ...

    // debounce
    if (candidateLabel != currentLabel && millis() - eventStartTimestamp >= MIN_EVENT_DURATION_MS) {
        currentLabel = candidateLabel;
        eventStartTimestamp = millis();
    }

    ...
}
```

Priority: *MAGNETIC_DISTURBANCE_EVENT* > *BREATH_OR_WARM_AIR_EVENT* > *LIGHT_OR_COLOR_CHANGE_EVENT* > *BASELINE_NORMAL*



The screenshot shows a Serial Monitor window titled "Output Serial Monitor x". The message box contains the following text:

```
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/ttyACM0')

raw,rh=54.97,temp=26.50,mag=38.523,r=8,g=8,b=7,clear=22
flags,humid_jump=0,temp_rise=0,mag_shift=0,light_or_color_change=0
event,BASELINE_NORMAL

raw,rh=54.97,temp=26.52,mag=39.497,r=8,g=8,b=7,clear=22
flags,humid_jump=0,temp_rise=0,mag_shift=0,light_or_color_change=0
event,BASELINE_NORMAL

raw,rh=54.97,temp=26.52,mag=34.540,r=8,g=8,b=7,clear=22
flags,humid_jump=0,temp_rise=0,mag_shift=0,light_or_color_change=0
event,BASELINE_NORMAL
```

Figure 24: Serial Monitor for Task 11, *BASELINE_NORMAL*.

```
Output  Serial Monitor x
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/ttyACM0')

raw,rh=73.89,temp=26.93,mag=36.346,r=0,g=0,b=0,clear=0
flags,humid_jump=1,temp_rise=0,mag_shift=0,light_or_color_change=0
event,BREATH_OR_WARM_AIR_EVENT

raw,rh=74.68,temp=27.03,mag=36.083,r=0,g=0,b=0,clear=0
flags,humid_jump=1,temp_rise=0,mag_shift=0,light_or_color_change=0
event,BREATH_OR_WARM_AIR_EVENT

raw,rh=75.28,temp=27.10,mag=36.892,r=0,g=0,b=0,clear=0
flags,humid_jump=1,temp_rise=0,mag_shift=0,light_or_color_change=0
event,BREATH_OR_WARM_AIR_EVENT
```

Figure 25: Serial Monitor for Task 11, *BREATH_OR_WARM_AIR_EVENT*.

```
Output  Serial Monitor x
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/ttyACM0')

raw,rh=51.42,temp=28.03,mag=59.883,r=6,g=5,b=6,clear=19
flags,humid_jump=0,temp_rise=0,mag_shift=1,light_or_color_change=0
event,MAGNETIC_DISTURBANCE_EVENT

raw,rh=51.42,temp=28.03,mag=59.883,r=6,g=5,b=6,clear=19
flags,humid_jump=0,temp_rise=0,mag_shift=1,light_or_color_change=0
event,MAGNETIC_DISTURBANCE_EVENT

raw,rh=51.41,temp=28.00,mag=58.771,r=6,g=5,b=5,clear=17
flags,humid_jump=0,temp_rise=0,mag_shift=1,light_or_color_change=0
event,MAGNETIC_DISTURBANCE_EVENT
```

Figure 26: Serial Monitor for Task 11, *MAGNETIC_DISTURBANCE_EVENT*.

```
Output  Serial Monitor x
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/ttyACM0')

raw,rh=54.92,temp=26.87,mag=52.697,r=40,g=56,b=154,clear=407
flags,humid_jump=0,temp_rise=0,mag_shift=1,light_or_color_change=1
event,LIGHT_OR_COLOR_CHANGE_EVENT

raw,rh=54.82,temp=26.87,mag=51.352,r=23,g=22,b=42,clear=95
flags,humid_jump=0,temp_rise=0,mag_shift=1,light_or_color_change=1
event,LIGHT_OR_COLOR_CHANGE_EVENT

raw,rh=54.73,temp=26.87,mag=51.196,r=23,g=22,b=42,clear=95
flags,humid_jump=0,temp_rise=0,mag_shift=1,light_or_color_change=1
event,LIGHT_OR_COLOR_CHANGE_EVENT
```

Figure 27: Serial Monitor for Task 11, *LIGHT_OR_COLOR_CHANGE_EVENT*.